



FACULDADE DE TECNOLOGIA SENAI GASPAR RICARDO JÚNIOR

SISTEMA DE SUPERVISÃO INDUSTRIAL VIA WEB COM OPC UA

WEB-BASED INDUSTRIAL SUPERVISION SYSTEM WITH OPC UA

Pedro Vieira Cavalciuk

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema acadêmico de supervisão industrial via web, voltado ao monitoramento de uma bancada simulada por meio do protocolo OPC UA. A solução integra um servidor OPC UA em Python, uma camada de comunicação por bridge Python ou Node-RED, um backend em Node.js com Express e uma interface web em Next.js/React. O sistema permite acompanhar variáveis como temperatura, pressão, umidade, estado de operação e trava de segurança, disponibilizando os dados em um painel supervisório acessível pelo navegador. A metodologia envolveu a simulação de sinais industriais, a coleta e normalização dos dados, a exposição das informações por API HTTP e a visualização periódica no frontend. Os resultados demonstram a viabilidade da integração entre tecnologias de automação industrial e desenvolvimento web para fins didáticos, laboratoriais e de validação de conceito.

Palavras-chave: OPC UA; Supervisão Industrial; Node.js; Next.js; Indústria 4.0.

ABSTRACT

This paper presents the development of an academic web-based industrial supervision system designed to monitor a simulated bench through the OPC UA protocol. The solution integrates a Python OPC UA server, a communication layer

using either a Python bridge or Node-RED, a Node.js backend with Express, and a Next.js/React web interface. The system enables the monitoring of variables such as temperature, pressure, humidity, operating status, and safety lock, displaying the data in a supervisory dashboard accessible through a web browser. The methodology involved the simulation of industrial signals, data collection and normalization, exposure of information through an HTTP API, and periodic visualization on the frontend. The results demonstrate the feasibility of integrating industrial automation technologies with modern web development for educational, laboratory, and proof-of-concept purposes.

Keywords: OPC UA; Industrial Supervision; Node.js; Next.js; Industry 4.0.

1 Introdução

A digitalização de processos industriais tem ampliado a necessidade de ferramentas capazes de integrar sinais de campo, sistemas de automação e aplicações web. Nesse cenário, a supervisão industrial deixa de depender apenas de interfaces locais e passa a considerar soluções conectadas, acessíveis por navegador e capazes de apresentar informações operacionais de forma organizada.

Este trabalho apresenta um sistema acadêmico de supervisão industrial via web, desenvolvido para monitorar uma bancada simulada por meio do protocolo OPC UA. O projeto busca aproximar conceitos de automação industrial, integração entre tecnologia da informação e tecnologia operacional e desenvolvimento web moderno. Em termos práticos, a aplicação monitora temperatura, pressão, umidade, estado de operação e trava de segurança.

O objetivo principal é demonstrar a viabilidade técnica de coletar dados simulados de uma planta industrial, tratá-los em uma camada de backend e disponibilizá-los em uma interface visual. A solução foi estruturada de forma didática, com componentes separados e funções bem definidas, permitindo compreender o fluxo completo de aquisição, transporte, padronização, exposição via API e visualização supervisória.

2 Revisão de Literatura

A Indústria 4.0 é marcada pela integração entre sistemas físicos, redes de comunicação, análise de dados e aplicações digitais. Nesse contexto, protocolos industriais padronizados são fundamentais para permitir que equipamentos, sensores e softwares troquem informações de maneira confiável e interoperável.

Segundo a OPC Foundation (2025), o OPC UA é um padrão de troca de informações voltado para comunicações industriais seguras, confiáveis, independentes de fabricante e de plataforma. Essa característica torna o protocolo adequado para cenários em que diferentes sistemas precisam compartilhar dados operacionais.

O Node.js é um ambiente de execução JavaScript multiplataforma utilizado para a criação de servidores, aplicações web, ferramentas de linha de comando e scripts (NODE.JS, 2026). Em conjunto com o Express, que é descrito como um framework minimalista e flexível para aplicações web e APIs (EXPRESS, 2026), ele possibilita a construção de camadas intermediárias para receber, tratar e disponibilizar dados.

No frontend, o Next.js é apresentado como um framework React para construção de aplicações web full-stack, utilizando componentes React para criação de interfaces

e recursos adicionais para otimização e organização do desenvolvimento (VERCEL, 2026). O React, por sua vez, fornece recursos como componentes e hooks, utilizados para sincronizar interfaces com sistemas externos e atualizar dados de forma dinâmica (META, 2026).

Outra ferramenta relevante em protótipos de integração é o Node-RED, definido pela OpenJS Foundation como uma plataforma low-code para aplicações orientadas a eventos, com foco em coletar, transformar e visualizar dados em tempo real (OPENJS FOUNDATION, 2026). Dessa forma, sua utilização em projetos acadêmicos facilita a criação de fluxos de comunicação entre dispositivos, APIs e serviços web.

3 Metodologia

O sistema foi desenvolvido em quatro blocos principais. O primeiro bloco corresponde ao servidor OPC UA em Python, responsável por simular uma bancada industrial. Esse servidor publica as variáveis Temperature, Pressure, Running, Humidity e SafetyLock, alterando os valores automaticamente a cada dois segundos para criar um ambiente contínuo de testes.

O segundo bloco é a camada de integração. Nessa etapa, foram previstas duas estratégias: uma bridge.py em Python e um fluxo alternativo em Node-RED. A bridge atua como cliente OPC UA, lê as variáveis do servidor e envia um pacote consolidado para o backend por meio do endpoint /iot. Já o Node-RED permite uma estratégia visual, publicando leituras individuais por tag e valor.

O terceiro bloco é o backend em Node.js com Express. Essa camada recebe os dados, normaliza os nomes dos campos, converte tipos de valores, identifica o dispositivo, calcula o estado de conectividade e mantém um histórico reduzido das leituras mais recentes. O backend não utiliza banco de dados, mantendo os dados em memória para reforçar o caráter didático do projeto.

O quarto bloco é o frontend em Next.js/React, desenvolvido como painel supervisor. A interface realiza consultas periódicas ao backend, exibe o número de dispositivos conectados, indica dispositivos sem comunicação e apresenta cartões com os principais valores monitorados.

4 Resultados e Discussões

O sistema desenvolvido apresentou funcionamento adequado para a proposta acadêmica. A simulação OPC UA permitiu gerar dados variáveis de sensores e estados operacionais, enquanto a camada de integração realizou o envio das leituras para o backend. A existência de duas opções de comunicação, bridge

Python e Node-RED, ampliou o valor didático do projeto, pois demonstrou tanto uma abordagem programática quanto uma abordagem visual de integração.

O backend mostrou flexibilidade ao aceitar payloads consolidados e leituras individuais por tag. Isso é importante em ambientes industriais, nos quais sistemas diferentes podem enviar dados em formatos variados. Além disso, a identificação de conectividade do dispositivo permitiu distinguir equipamentos online de equipamentos sem comunicação, com base no tempo da última atualização.

No frontend, os dados foram apresentados em cartões organizados, destacando estado operacional, última leitura, conectividade e valores dos sensores. Também foram tratados cenários de indisponibilidade do backend e ausência de dados, o que contribui para uma experiência de uso mais robusta.

4.1 Exemplo de Tabela

Tabela 1: Variáveis monitoradas pelo sistema de supervisão industrial.

Variável	Descrição	Origem
Temperature	Temperatura simulada da bancada	Servidor OPC UA
Pressure	Pressão simulada do processo	Servidor OPC UA
Humidity	Umidade simulada do ambiente	Servidor OPC UA
Running	Estado de operação do sistema	Servidor OPC UA
SafetyLock	Trava de segurança calculada por condição operacional	Servidor OPC UA

Como limitação, observa-se que o sistema não utiliza banco de dados, não possui autenticação, não implementa comunicação em tempo real por WebSocket e mantém o estado temporariamente em memória. Essas características não comprometem a proposta, pois o projeto foi desenvolvido como plataforma de demonstração, ensino e validação de conceito. Para trabalhos futuros, recomenda-se adicionar persistência em banco de dados, histórico gráfico, alarmes, autenticação de usuários, suporte a múltiplos dispositivos reais e comunicação em tempo real.

5 Conclusão

O projeto demonstrou a viabilidade de integrar uma bancada industrial simulada a uma aplicação web supervisória, utilizando OPC UA, Node.js, Express, Next.js, React e Node-RED. A solução permitiu acompanhar variáveis de processo, estado operacional e conectividade, representando de forma didática um fluxo compatível com cenários de digitalização industrial.

A arquitetura adotada evidenciou a separação de responsabilidades entre simulação, integração, backend e frontend. Essa organização facilita a compreensão do sistema e permite futuras expansões, como armazenamento histórico, alarmes, autenticação e uso de dispositivos reais. Assim, o trabalho contribui para o aprendizado prático sobre integração IT/OT, supervisão industrial e aplicações web voltadas à Indústria 4.0.

Agradecimentos

Agradecemos à instituição de ensino, aos professores orientadores e aos colegas que contribuíram para o desenvolvimento e validação deste projeto acadêmico.

Referências

EXPRESS. Express: Node.js web application framework. Disponível em: <https://expressjs.com/>. Acesso em: 18 maio 2026.

META. React: Built-in React Hooks. Disponível em: <https://react.dev/reference/react/hooks>. Acesso em: 18 maio 2026.

NODE.JS. Run JavaScript Everywhere. Disponível em: <https://nodejs.org/en>. Acesso em: 18 maio 2026.

OPC FOUNDATION. OPC Unified Architecture. Disponível em: <https://opcfoundation.org/opc-ua/>. Acesso em: 18 maio 2026.

OPENJS FOUNDATION. Node-RED: Low-code programming for event-driven applications. Disponível em: <https://nodered.org/>. Acesso em: 18 maio 2026.

VERCEL. Next.js Docs. Disponível em: <https://nextjs.org/docs>. Acesso em: 18 maio 2026.